Assignment-Module-50
IO Operations

Q.1 Ans: A stream can be defined as a sequence of data. The InputStream is used to read data from a source andthe OutputStream is used for writing data to a destination.

Q2 Ans:- L write() - writes the specified byte to the output streamO
L write(byte[] array) - writes the bytes from the specified array to the output streamO
L flush() - forces to write all data present in the output stream to the destinationO
L close() - closes the output stream.

Q3 Ans:-Serialization is the process of converting an object into a stream of bytes to transfer it over a network or to store it in a file or database. In Java, serialization is done by implementing the Serializable interface.

Q4. Ans: The Serializable interface in Java is a marker interface that has no methods. It is used to mark classes that can be serialized, meaning their object instances can be converted into a stream of bytes.

Q5.Ans: Deserialization is the process of converting a stream of bytes back into an object instance. This is done after an object has been serialized.

Q6. Ans: Serialization is achieved in Java by implementing the Serializable interface. When an object is serialized, its state is converted into a stream of bytes, which can then be transferred

over a network or stored in a file or database**.**

Q.7 Ans: Deserialization is achieved in Java by reading a stream of bytes and using them to recreate the origina lobject instance. This is done by calling the readObject() method of an ObjectInputStream instance.

Q.8Ans: Mark member variables as static or transient, and those member variables will no more be a part of Serialization.

Q.9Ans: The following classes are available in the Java IO API and are important to work with files in Java.
File
RandomAccessFile
FileInputStream
FileReader
FileOutputStream
FileWriter

Q.10 Ans: The `Externalizable` and `Serializable` interfaces in Java are used to define how an object can be serialized (converted into a stream of bytes) and deserialized (restored back to an object). However, there are significant differences in how they work and their purpose.

Key Differences:

Feature	Serializable	Externalizable

Definition	A marker interface (does not contain any methods) used to enable default serialization.	An interface that extends `Serializable` and provides custom serialization methods.
Methods to Implement	No methods to implement; relies on default JVM-provided serialization logic.	Requires implementing two methods: - `void writeExternal(ObjectOutput out)` - `void readExternal(ObjectInput in)`
Control Over Serialization	No control over the serialization process. The default process serializes all non-transient fields automatically.	Complete control over the serialization process, allowing you to explicitly define what to serialize and deserialize.
Default Serialization	Yes, all non-transient and non-static fields are serialized automatically.	No, the programmer must explicitly implement the serialization logic.
Performance	Slower, as it uses reflection and handles the entire object structure automatically.	Faster, as the serialization logic is manually controlled and more efficient.
Customization	Limited customization by using `transient` keyword or overriding `writeObject()` and `readObject()` methods.	Full customization, as you explicitly define the logic for serialization and deserialization.
Serializable Version UID	The `serialVersionUID` is optional but recommended for versioning consistency.	The `serialVersionUID` is mandatory, as the process is manual.
Use Case	Suitable for simple and default serialization needs.	Suitable for custom serialization requirements where performance and control are critical